

CQt day1 hw3 레포트

로봇 예비단원 주호진

1. 과제

플레이어 클래스, 몬스터 클래스를 각각 생성하고 플레이어가 몬스터를 사냥하는 스크립트를 작성.

명령어를 입력 받아 플레이어가 이동, 공격, 스테이터스 표기를 실행

공격을 해서 몬스터의 HP가 0이 되거나 MP가 부족한 상태에서 공격한다면 프로그램 종료(공격당 MP 1 소모)

```
Type Command(A/U/D/R/L/S)
D
Y Position -1 moved!
Type Command(A/U/D/R/L/S)
R
X Position 1 moved!
Type Command(A/U/D/R/L/S)
S
HP:50
MP:10
Position:1,-1
Type Command(A/U/D/R/L/S)
U
Y Position 1 moved!
Type Command(A/U/D/R/L/S)
D
Y Position -1 moved!
Type Command(A/U/D/R/L/S)
L
X Position -1 moved!
Type Command(A/U/D/R/L/S)
S
HP:50
MP:10
Position:0,-1
Type Command(A/U/D/R/L/S)
A
공격 실패!
Type Command(A/U/D/R/L/S)
A
공격 실패!
Type Command(A/U/D/R/L/S)
R
HP:50
MP:8
Position:0,-1
Type Command(A/U/D/R/L/S)
```

명령어를 입력 받아 플레이어가 이동, 공격, 스테

```
Type Command(A/U/D/R/L/S)
A
공격 성공!
남은 체력:40
Type Command(A/U/D/R/L/S)
A
공격 성공!
남은 체력:30
Type Command(A/U/D/R/L/S)
A
공격 성공!
남은 체력:20
Type Command(A/U/D/R/L/S)
A
공격 성공!
남은 체력:10
Type Command(A/U/D/R/L/S)
A
공격 성공!
남은 체력:0
Monster Die!!
```

```
Type Command(A/U/D/R/L/S)
S
HP:50
MP:0
Position:0,-1
Type Command(A/U/D/R/L/S)
A
MP 부족!
...Program finished with exit c
Press ENTER to exit console.
```

공격을 해서 몬스터의 HP가 0이 되면 프로그램
격 한 번당 MP 1소모, MP가 부족한 상태에서 공

2. 코드 설명

파일은 class 헤더파일, class 소스파일, 메인파일로 나누었다.

2-1. Class 헤더파일

```
h++ hw3.hpp M X C++ hw3_main.cpp M X C++ hw3.cpp M X
C++&Qt > day1 > hw3 > hw3_package > include > h++ hw3.hpp > ...
1  class Monster
2  {
3      public:
4          int HP, x, y;
5          Monster();
6          Monster(int x, int y, int HP);
7          int Be_Attacked();
8  };
9
10 class Player
11 {
12     public:
13         int HP, MP, x, y;
14         Player();
15         Player(int x, int y);
16         void Attack(Monster &target);
17         void Show_status();
18         void X_move(int move);
19         void Y_move(int move);
20 };
21 |
```

Class 헤더 파일은 ppt에 제시된 대로 사용하였다.

2-2. Class 구조파일

```
h++ hw3.hpp M x C++ hw3_main.cpp M x C++ hw3.cpp M x
C++&Qt > day1 > hw3 > hw3_package > src > C++ hw3.cpp > Show
1  #include "../include/hw3.hpp"
2  #include <iostream>
3
4  using namespace std;
5
6  Monster::Monster(int x, int y, int HP)
7  {
8      this->x = x;
9      this->y = y;
10     this->HP = HP;
11 }
12 Player::Player(int x, int y)
13 {
14     this->x = x;
15     this->y = y;
16 }
17
```

Class 구조파일에는 헤더파일에 선언된 함수와 생성자의 정의를 넣었다.

```
void Player::Show_status()
{
    cout << "HP: " << HP << endl;
    cout << "MP: " << MP << endl;
    cout << "Position: " << x << ", " << y << endl;
}

void Player::X_move(int move)
{
    x = x + move;
    cout << "X Position " << move << " moved!" << endl;
}

void Player::Y_move(int move)
{
    y = y + move;
    cout << "Y Position " << move << " moved!" << endl;
}
```

위 세개의 함수는 각각 HP, MP, 위치를 표시해주는 함수, X 이동 함수, Y 함수이다.

```

28
29 void Player::Attack(Monster &target)
30 {
31
32     // 좌표 맞고 MP 있을 때, MP 부족일 때, 그 외로 분류
33     if (target.x == x && target.y == y && MP > 0)
34     {
35         MP = MP - 1;
36         cout << "공격 성공!" << endl;
37         target.Be_Attacked();
38     }
39     else
40     {
41         MP = MP - 1;
42         cout << "공격 실패!" << endl;
43     }
44 }

```

Attack 함수는 좌표가 동일하면서 MP가 있을 때 공격이 성공하도록 설정하였고, 그 외에는 MP만 소모되고 공격되지 않게 하였다. MP가 0인 경우는 main문에서 처리해주었다.

```

int Monster::Be_Attacked()
{
    HP = HP - 10;
    cout << "남은 체력: " << HP << endl;

    if (HP <= 0) // 10이면 사망
        return 1;
    else
        return 0;
}

```

Attack 함수에서 조건을 만족했을 때 몬스터의 체력이 10씩 까이게 했다. Ppt에서 제시된 자료에서 해당 함수 반환형이 int형이어서 이를 이용하려 했으나 main문에서 처리하게 되었다. 그 이유는 뒤에 main 파일 설명에서 기재하겠다.

2-3. Main 파일

```

16 #include "../include/hw3.hpp"
17 #include <iostream>
18
19 int main()
20 {
21     Player player(0, 0);
22     Monster monster(1, 1, 50);
23
24     player.HP = 50, player.MP = 10;
25
26     char ch;
27     int is_end = 0;
28

```

Ppt에서 제시된 초기값을 설정하고, 예시문에서 player의 기본 체력과 MP가 50과 10인 것 같아서 추가로 설정하였다. 사용자에게 입력받을 변수와, 게임이 종료되는 flag를 판별하기 위해 is_end를 변수로 설정했다.

```

while (1)
{
    std::cout << "Type Command(A/U/D/R/L/S)" << std::endl;
    std::cin >> ch;
}

```

기본적으로 계속 루프가 돌기 때문에 while문으로 설정해주었고 ppt에 나와있는 대로 text와 커맨드 하나를 입력받도록 했다.

```

switch (ch)
{
case 65: // A
    if (player.MP == 0)
    {
        is_end = 1;
        break;
    }
    player.Attack(monster);
    if (monster.HP == 0)
    {
        is_end = 1;
        break;
    }
    break;

case 85: // U
    player.Y_move(1);
    break;

case 68: // D
    player.Y_move(-1);
    break;

case 82: // R
    player.X_move(1);
    break;

case 76: // L
    player.X_move(-1);
    break;

case 83: // S
    player.Show_status();
    break;

default:
    break;
}

```

Switch문을 통해서 각각의 함수를 실행하도록 했고, Attack의 경우 실행하기 전 현재 MP를 확인해서 공격을 실행할지 말지 판별하게 된다. 원래는 int형 반환 함수인 Be_Attacked를 사용하려 했으나 void형인 Attack에서 실행되기 때문에 return을 받아오는 것보다 main에서 판별하는 것이 더 쉽다고 생각하여 이렇게 짜게 되었다.

프로그램이 종료되는 조건은 2개, 하나는 MP가 0인 상태에서 공격, 또 하나는 monster의 체력이 0이 될 때이므로 해당 상황을 인식하여 is_end의 변수값을 1로 올려 준다.

```
        if (is_end == 1)
            break;

        std::cout << std::endl;
    }

    return 0;
}
```

Switch 문을 빠져나왔을 때 is_end가 1이 됐다면 프로그램을 종료하고, 1이 아닌 경우에는 계속 루프를 돌게 되어있다.

3. 실행 결과 화면

3-1. MP가 0인 상태에서 종료한 경우

Type Command(A/U/D/R/L/S)

A

공격 실패 !

Type Command(A/U/D/R/L/S)

A

공격 실패 !

Type Command(A/U/D/R/L/S)

A

공격 실패 !

Type Command(A/U/D/R/L/S)

A

공격 실패 !

Type Command(A/U/D/R/L/S)

A

공격 실패 !

Type Command(A/U/D/R/L/S)

A

공격 실패 !

Type Command(A/U/D/R/L/S)

A

공격 실패 !

Type Command(A/U/D/R/L/S)

A

공격 실패 !

Type Command(A/U/D/R/L/S)

A

공격 실패 !

Type Command(A/U/D/R/L/S)

S

HP: 50

MP: 1

Position: 0, 0

Type Command(A/U/D/R/L/S)

A

공격 실패 !

Type Command(A/U/D/R/L/S)

S

HP: 50

MP: 0

Position: 0, 0

Type Command(A/U/D/R/L/S)

A

* 터미널이 작업에서 다시

3-2. Monster의 체력이 0이 된 경우

```
● [*] 작업 실행 중 : cd "/home/hojin/robit/rob...
e" && ./hw3_main.exe

Type Command(A/U/D/R/L/S)
U
Y Position 1 moved!

Type Command(A/U/D/R/L/S)
R
X Position 1 moved!

Type Command(A/U/D/R/L/S)
A
공격 성공 !
남은 체력 : 40

Type Command(A/U/D/R/L/S)
A
공격 성공 !
남은 체력 : 30

Type Command(A/U/D/R/L/S)
A
공격 성공 !
남은 체력 : 20

Type Command(A/U/D/R/L/S)
A
공격 성공 !
남은 체력 : 10

Type Command(A/U/D/R/L/S)
S
HP: 50
MP: 6
Position: 1, 1

Type Command(A/U/D/R/L/S)
A
공격 성공 !
남은 체력 : 0
[*] 터미널이 작업에서 다시 사용됩니다. 닫
```

Ppt에서는 몬스터의 좌표가 5, 4로 제시되어 있으나 빠른 디버깅을 위해 임의로 1, 1로 정하였다.

4. AI 사용 보고서

<https://share.gemini.google/7ZWcTxTAQnJJ>

cpp class에서 매개변수가 있는 생성자를 main문에 넣었는데
컴파일러가 인식하지 못하는 이유가 뭘까

C++ 컴파일러가 매개변수가 있는 생성자를 인식하지 못하거나 관련 에러를 발생시키는
원인은 주로 기본 생성자의 부재, 괄호 해석 오류, 접근 제어자 설정, 또는 인자 타입
불일치에서 비롯됩니다.

기본 생성자(Default Constructor) 자동 생성 중단

클래스에 사용자가 매개변수가 있는 생성자를 하나라도 정의하면, 컴파일러는 더 이상
기본 생성자(매개변수가 없는 형태)를 자동으로 만들어주지 않습니다. 이 상태에서
main 함수 등에서 `ClassName obj;` 처럼 인자를 전달하지 않고 객체를 생성하려고
하면 컴파일러는 일치하는 생성자가 없다고 판단하여 에러를 발생시킵니다.

- **해결책:** 매개변수가 있는 생성자와 함께 기본 생성자도 명시적으로 선언(
`ClassName() = default;` 등)하거나 필요한 인자를 빠짐없이 전달해야
합니다.

```
class Player
{
public:
int HP, MP, x, y;
Player();
Player(int x, int y);
}
```

이 class를 main에서 `Player p(0, 0)`하면 인식해야 하는 거
아니야?

클래스 선언 끝에 **세미콜론(;)**이 누락되어 컴파일러가 코드를 올바르게 읽어내지 못하기 때문입니다.

C++에서는 클래스나 구조체 선언이 끝날 때 반드시 중괄호 뒤에 세미콜론을 붙여야 합니다. 작성하신
코드는 다음과 같이 수정되어야 합니다.

생성자의 개념이 익숙하지 않아서 생성자 정의를 선언하지 않고 처음에 실행을 눌렀는데

실행되지 않아서 LLM을 사용했다. 생성자의 x와 해당 정의에서 받아오는 매개변수 x가 동일한데 이의 경우 this.를 붙여서 구분해준다는 것을 알았다.

void 함수 내에서 if문에 따라 int형 함수가 실행되는데 이 return을 받으려면 어떻게 해야 돼

int 형 함수의 호출 결과를 받을 수 있는 변수를 선언하고 대입하면 됩니다. void 함수 내부라 하더라도 반환값을 변수에 저장하거나 조건식 안에서 바로 활용할 수 있습니다.

C++

```
int getBonusScore() {  
    return 50;  
}  
  
void processPlayer(bool isBonusActive) {  
    if (isBonusActive) {  
        int score = getBonusScore(); // 여기서 int형 함수의 return 값을 받습니다!  
        // score 변수를 활용한 로직 처리  
    }  
}
```

이건 위 코드에서 언급된 부분인데 void 함수인 Attack에서 Be_Attacted가 실행될 때 해당 int형의 반환값을 어떻게 받아오는지 질문한 부분이다. 근데 과제를 수행하는 시간이 부족하여서 일단 급하게 main문에서 따로 확인을 해주는 방식으로 구현하였다.